Olmran Item Builder

Full Usage Guide - v6.4.0

A tool for parsing Olmran/MUD-style game logs, extracting loot data, building a master equipment database, and finding the optimal gear set for your character.

This guide covers every tab and feature currently in the program. It's a companion to the README on GitHub, written for reading start-to-finish or jumping to the section you need.

Contents

1. Getting Started

- Two download options on the download page: **OlmranItemBuilder.exe** (single file, simplest) or **OlmranItemBuilder_Folder.zip** (unzip once, then run the exe from inside the extracted folder - keep the whole folder together). Same program either way.
- Put it anywhere on your computer, then double-click the exe to run - no Python install, no admin rights, no setup step.
- Windows SmartScreen may warn about an unrecognized app the first time (it isn't code-signed yet) - click "More info" then "Run anyway".
- The single-file download is a PyInstaller "onefile" build, so it unpacks itself to a temp folder on every launch - expect a few seconds of startup delay each time, not just the first. The Folder download has no such delay (nothing to unpack on launch) since everything's already sitting in the folder - prefer it if the onefile startup time bothers you, or if you've ever seen a "Failed to load Python DLL" error.
- The bundled community equipment list (*Olmran_Community_Eq_and_Stats_List.xlsx*) ships inside the download itself either way and loads automatically on startup.
- A small bar at the very top of the window (visible on every tab) has two buttons: **Download Page** opens this download site in your browser, and **Check for Update** checks GitHub's latest release against your current version. If you're current, it just says so; if a newer version exists, the button itself becomes **Download & Update** - click it (after confirming) to download and install the new version, no manual reinstall needed. This correctly fetches whichever flavor (onefile or Folder) you're already running. The app closes to install it - a message lets you know when it's done, and you just open the app again yourself afterward.

2. Parse Tab

Load your game logs here and turn them into structured Chat, Combat, and Loot data.

- Load CHAT logs and ACTION logs - the file type is auto-detected from the filename. The Parse tab has its own inner sub-tabs: **Files & Search** (everything below), **Counters**, and **Settings** - the file list appears on both Files & Search and Counters and stays in sync, so files can be loaded without leaving either one.
- **Counters** (its own sub-tab) - one shared "Load Log Files" section up top, with **XP Counter**, **Damage Counter**, and **PvP Damage** as tabs below it:
- **XP Counter** - computes overall XP/hour from the loaded log(s), parsed from "You gain BASE (+BONUS) experience points." lines, plus a **Per-Area Summary** breakdown using time actually spent in each area (tracked via the game's own room-title lines), and an **Every Gain** tab listing every individual XP gain (Timestamp/Area/XP Gained/File).

- **Damage Counter** - totals damage dealt to every mob across the loaded log(s), with a **Per-Mob Summary** (Mob/Damage/Hits/Avg per Hit) and an **Every Hit** tab listing every individual hit (Timestamp/Mob/Damage/Weapon/File) - sorting Every Hit by Mob groups every hit against one mob together, useful for isolating damage-testing hits on a specific mob without singling it out during logging. The **Weapon** column shows the melee weapon used, the actual spell cast, or - for a weapon's own innate elemental proc with no cast line - the weapon that triggered it. A hit landing on a player instead of a mob is excluded here entirely - see PvP Damage below.
- **PvP Damage** - splits player-vs-player combat into **Damage Dealt** (hits you land on another player) and **Damage Taken** (hits another player lands on you), each with its own Per-Player Summary + Every Hit tables, same layout and columns as the mob Damage Counter. A target is recognized as a player rather than a mob by name shape alone - a bare single word with no leading "a/an/the" article (mob names almost always carry one, e.g. "a tortured spirit"), matching how the game's own combat text distinguishes the two.
- **Item Counter** - drop rate of every item, per mob, across the loaded log(s), broken down by Zone and by day/week/month/year:
- **Overall** tab is a nested tree, **Zone** → **Mob** → **Item** - expand a zone to see every mob killed there, expand a mob to see its own Item/Drops/Kills/Drop Rate % rows. Zone and Mob header rows show their own summed Drops/total Kills/blended Drop Rate % across whatever's nested under them. **Daily**/**Weekly**/**Monthly**/**Yearly** are each set up the exact same way, with Period as one more level on top (bucketed by each log file's own OS creation date, not anything parsed from the filename) - only meaningfully populated once the loaded logs' creation dates span more than one of that period. A **Log Files** tab lists every loaded file's own extracted date and kill/drop counts.
- **Drop Rate %** = (drops ÷ kills × 100) ÷ possible items - possible items being how many distinct items the loaded master/community database lists for that mob (1 if the mob isn't in the database at all), so an item competing against a bigger loot table reads as rarer even at the same drops/kills ratio, rather than every item under a mob summing to a plain 100%. The exact formula is spelled out as a note next to the tab's own Save/Delete button, rather than crammed into the column header.
- Right-click a **Zone** row for **Open into new tab** (that zone's own Mob/Item/Drops/Kills/Drop Rate % table), a **Mob** row for **Open monster in new tab** (every zone it was killed in and what it dropped there), or an **Item** row for **Open item in new tab** - unlike Zone/Monster (one mini-tab per target, reused on repeat clicks), every item opened this way accumulates into the SAME shared "Item Comparison" mini-tab instead of spawning a new one each time, building up a custom watchlist to compare side by side. Every level also gets **Add to (name)** - appends that same selection into any already-open mini-tab you pick, instead of opening/reusing its own dedicated tab - and **Delete**, which removes just that one row from whichever table it was clicked in (display-only, doesn't touch the underlying computed data or any other open tab).
- Every mini-tab shows a trailing **✕** to close it - click the mark itself, near the tab's own right edge; clicking anywhere else on the tab (its label text, or "All", which is never closable) just selects it like any other tab. A blank line automatically separates rows belonging to different zones in any table that spans more than one (a Monster tab naturally covering every zone that mob appeared in, an Item Comparison item seen in more than one zone, or rows brought in via "Add to").
- **Items** / **Junk Loot** checkboxes (Items on, Junk Loot off by default) - Items means "found in the loaded master/community database"; Junk Loot is everything else. Applies the next time you click Item Counter.

- **Character** dropdown - defaults to "All Characters"; repopulated from whatever's currently loaded, extracted from each log filename's own character-name segment (e.g. "ACTION-PDF-GNAWBIE-1785249871654.log" → "GNAWBIE" - the segment right before the trailing timestamp). A file whose name doesn't fit that pattern counts as "Unknown". Picking one character scopes everything (Overall, every period breakdown, and the Consistent tally below) to just that character's own logs.
- **Consistent** checkbox (checked by default) - keeps a running tally per character across every past click instead of starting over from just what's currently loaded; any log file already counted for that character is skipped entirely rather than re-parsed and double-counted, so reloading the same logs across sessions is always safe. "All Characters" then shows every character's own persisted tally combined; picking one character shows just its own. Unchecking it (a confirmation popup asks first, since it's easy to click by accident) reverts to a plain one-off computation over whatever's currently loaded, same as if this persistence didn't exist.
- A bracket line like "[Sin 60]" - a class/summon status display, its name padded with 3 or more spaces before a trailing level/HP number - is recognized as not a real zone transition and excluded from every Zone-based breakdown; its kills/drops still count toward that mob's Overall totals, they just aren't attributed to any zone.
- Same as the other three Counters: sortable columns, and a **Save** button to turn a result into its own persistent, named tab with **Delete**, surviving a restart.
- All four counters' tables sort by clicking any column header, and all support a **Save** button to turn a result into its own persistent, named tab with a **Delete** button, surviving a restart. Every Gain and Every Hit rows also support right-click **Open Log at This Line**, same as Search Logs and PvP results below.
- This sub-tab's own "Load Log Files" table is drag-resizable from its bottom edge (a small handle just below it) - shrink it to give more room to the XP/Damage/PvP Damage/Item Counter notebook below. Ranges from 8 lines (its original size, the maximum) down to 3, starting at 4 by default; the size you leave it at persists across restarts. Files & Search's own copy of this table is unaffected and stays fixed at 8.
- **Search Logs** finds every time an item actually dropped across a whole batch of loaded logs, independent of running a full parse:
- **Drops only** (default, on) - matches real drop events the same way the Loot parser does, so quality/material prefixes ("bright", "glowing", "shining", etc.) don't cause misses or false confidence. For each drop it shows a **snapshot**: every line from the last timestamp seen through the drop line itself, so you can see the fight and kill that led up to it. Click a result row to view its snapshot.
- Uncheck **Drops only** to instead do a plain raw-text search of every loaded file's lines (any mention of the term, not just drops), showing filename, line number, and timestamp per match.
- Case-sensitive matching is optional in both modes.
- Right-click any result row (either mode) for **Open Log at This Line** - opens the original log file in a new window, scrolled to and highlighting that exact line, so you can see it in its complete original context. That window has its own Find box at the top (with up/down buttons, or plain Enter/Shift+Enter) to search around the rest of the file, wrapping around at either end.
- **PvP** - three searches for player-vs-player combat, each result row also supporting Open Log at This Line:

- **Your Kills** - just a button, no name needed. Finds every time you killed someone ("You just killed !" immediately followed by "You earn N realm points!"), each labeled "You Killed " with the name pulled straight out of the matched line. Next to the button, a **Total RP** field shows the sum of realm points earned across every matched kill, and a **Kills** field shows how many kills were found - both fill in automatically each time this search runs.
- **Your Deaths** - just a button, no name needed. Finds every time someone killed you (their killing blow line ending "at you for N damage!" immediately followed by a blank line and your own death message), each labeled " Killed you" with the attacker's name pulled straight out of the matched line. The "N minutes" at the end of the death message is unique per death and isn't part of the match. Dying earns no realm points, so there's no RP field here.
- **Participated** - just a button, no name needed. Finds every "You earn N realm points!" line that isn't immediately preceded by "You just killed !" (that combo is a direct kill, already covered by Your Kills) - credit for participating in someone else's kill without landing the final blow yourself. Same **Total RP**/**Participations** fields as Your Kills, next to its own Search button.
- Parse options (Chat / Combat / Loot) automatically enable or disable based on which file types are loaded. A second row of checkboxes - **XP Counter**, **Damage Counter**, **PvP Dealt**, **PvP Taken** - sits below it, independent of that auto-enable logic; check any of these and click Run Parse to compute that Counter's data (the same engine as the Counters sub-tab's own buttons) and make it available to Export.
- Use the Snapshot buttons to preview parsed data before exporting - a second row covers the four Counters (XP/Damage/PvP Dealt/PvP Taken).
- Export everything to a formatted spreadsheet or plain-text file - see Settings' own Export sub-tab below for the full set of formats and options.
- **Credits** - far right of the Parse Results row. Opens a popup listing everyone who contributed to the project.
- **Settings** (its own sub-tab) - a mini-notebook holding **Fields** and **Export**:
- **Fields** - customize which columns get extracted for each of 7 types: Chat, Combat, Loot, **XP Counter**, **Damage Counter**, **PvP Dealt**, and **PvP Taken**:
- Add, edit, remove, and reorder fields per data type.
- "Reset to Defaults" restores the built-in field set.
- **Export** - customizes and writes out whichever of Chat/Combat/Loot/the 4 Counters actually has data, using whatever columns were set up in Fields:
- **File Format** dropdown - Excel Workbook (.xlsx, the default), Excel 97-2003 (.xls), OpenDocument Spreadsheet (.ods), CSV, or TSV, with a one-line explanation of whichever is currently selected shown just below it. XLSX/XLS/ODS keep every checked type as a separate sheet in one file; CSV/TSV can't hold more than one table per file, so each checked type instead becomes its own file (named with that type as a suffix) - picking either of those two shows a folder picker instead of a single-file "Save As" dialog, since more than one file is always produced.
- Optional summary sheet with run statistics - available in every format, though only XLSX gets the fully styled version (bold title, colored header); the others get a plain label/value table with the same information.

- **Sheets to Export** - a checklist with one entry per Chat file/Combat/Loot/XP Counter/Damage Counter/PvP Dealt/PvP Taken that currently has data, each showing its row count, with Select All/Select None/Refresh buttons. This is what actually decides what gets written - independent of Parse Options' own checkboxes, which only control what gets computed in the first place. It refreshes automatically after Run Parse, and also after using any of the Counters sub-tab's own XP Counter/Damage Counter/PvP Damage Counter buttons directly (no need to run Parse a second time just to make that same data exportable).
- Two entirely separate destinations, each with its own Output filename and Export button - **Export Chat / Combat / Loot** (which also carries the Master Database section below) and **Export Counters** (a plain snapshot file for the 4 Counters, with no master-database/accumulation behavior - just this export's own data, easy to hand to someone without needing this program to view it). Each only ever writes sheets checked above that belong to its own group.
- Create a new Master Loot Database or append to an existing one (fodder items excluded automatically) - part of the Chat/Combat/Loot destination specifically, always a .xlsx file regardless of the File Format dropdown, since it's a separate persistent database, not this particular export's own output.
- Export Combat and Loot as separate files if needed - XLSX only, part of the Chat/Combat/Loot destination; disabled automatically for every other format (CSV/TSV are already inherently one-file-per-type, as above).

3. Build Tab

This is where you search a master equipment database for the best gear for your character. It's organized into three inner sub-tabs: Basic Constraints, Bank Build, and Manual. Basic Constraints itself further splits into its own mini-tabs - Basic, Armor Constraints, Weapon Constraints, and Edibles (sections 3.2-3.5 below) - rather than those being separate top-level sub-tabs. Min/Max/Specific Level and the search buttons (3.8) live inside Basic Constraints' own Wanted Spells column, so they're only visible there - Bank Build has its own per-character Find Best Bank Build button instead, and Manual has its own separate controls entirely (see 3.7).

3.1 Master Database File

- **Browse...** - pick your own .xlsx database.
- **Create New** - start a blank one.
- **Use Community List** - instantly loads the bundled community equipment list.
- **Load** - reads the selected file into memory for searching.

Items whose Area is "Class" are excluded from every search entirely at load time (they're role-specific class-quest rewards, not found in the open world) - see the Class Items field below for the one way to still use one deliberately.

3.2 Basic Constraints

The "Build Constraints" box near the top of this sub-tab is itself a small mini-notebook with 4 tabs - **Basic** (everything below, through Save Constraints), **Armor Constraints** (3.4), **Weapon Constraints** (3.5), and

Edibles (3.3). Switching between them doesn't reset or duplicate anything - it's the exact same underlying settings either way, just organized into tabs instead of separate top-level sub-tabs.

Desired Spells - pick spells from category dropdowns instead of typing them, each paired with a tier dropdown ((any), i, ii, iii):

- **Basic** - Agility, Bless, Dexterity, Constitution, Intelligence, Wisdom, Strength, Evade, Combat, Tough.skin
- **Shields/Buffs** - Protect, Blur, Shield, Vitalize, Regenerate, Bleed.resist, Disease.resist, Poison.resist
- **Class Specific** - 23 combat/skill-enhance spells (Reverb, Aura, Backstab, Bash, Berzerk, Crush, Direct, Double, Fired, Improve, Leathers, Lockpick, Mark, Martialarts, Maul, Melee, Pummel, Rake, Repair, Slash, Tap, Thrust, Track, Volley - none of these go above tier ii)
- **General Skills** - Platemail, Leathers, Thrust, Chaos.crush, Slash, All.weapons, Weapons, Climb, Hide, Jump, Swim, Percept, Sneak
- **Protects** - the 8 elemental/mental protects (Cold/Earth/Elemental/Fire/Lightning/Mental/Shock/Water), with their own minor/normal/improved tier labels instead of i/ii/iii

Some spells restrict which tiers are selectable (Agility/Bless cap at ii; Class Specific spells cap at ii; Disease.resist alone goes up to tier iv) - the tier dropdown greys out anything that spell can't reach. Click **Add to List** to add the current dropdown selection as a removable chip under **Wanted Spells**. Requesting the same spell at two different tiers (e.g. Dexterity i and Dexterity iii) is treated as one requirement, not two.

Wanted Sigils - pick one or more Sigil types (Cold/Earth/Fire/Lightning/Pain/Shock/Water) and the search actively favors armor carrying them, the same way it does for Wanted Spells - useful since many armor pieces carry a Sigil but no Spell at all. Only one item per Sigil type earns this preference across the whole build - once a Sigil's already been used somewhere, a slot with nothing else going for it (no Wanted Spell of its own) is left empty rather than getting a second, purely cosmetic copy of the same Sigil. A slot only ever reuses an already-claimed Sigil when doing so is the sole way to cover a genuine, otherwise-unreachable Wanted Spell - reuse for coverage's own sake is always fine, only the empty cosmetic case is excluded. A Wanted Spell always outranks a Wanted Sigil for the same slot regardless. Each chip also has two circles for a firmer ask than that passive preference: **red** requires one piece of gear carrying that sigil AND any Wanted/Priority spell; **blue** instead requires that sigil AND a matching Protect spell (that element's own, or the generic Elemental Protect), independent of the Wanted Spells list entirely. The two are mutually exclusive per sigil, and both are enforced even if it means leaving some other slot empty.

No wanted spell ever stacks across two places in the build - requesting the same base spell at two tiers (see above) is one requirement, not two, and this holds no matter which two slots (or an Edible, section 3.3) would otherwise both end up carrying it: only one of them ever actually does, and it's never overridden just because the other candidate happens to also carry a Wanted Sigil.

Priority - pick a spell and a tier (including "(any)"), then click the **+** button. Tier "(any)" always actively searches for that spell (not just preferred among items that already matched something else). A specific tier instead pairs that spell with that tier, so the search targets exactly it, even over a higher tier that's available elsewhere.

Required Items - type a specific item's name under **Require Item:** and click **Add to Build**. It's looked up by exact match first, falling back to a spelling-tolerant fuzzy search that asks you to confirm the closest match if there's no exact hit. Once added it's forced into its slot on every future search, with the rest of the build

calculated around it.

Class Items - a typeable field listing every item excluded for being in the "Class" area, formatted as *Item-MOB-Spell* and sorted by Mob (Mob shown in bold caps). Click or focus it to see the full list in a suggestion popup, or type to filter it by substring (same autocomplete style as the Area Items tab); pick an entry from the popup, then click **Add to Build** to force it into Required Items - the one deliberate way an otherwise-excluded item can still end up in a build.

Level Filtering - Min Level / Max Level restrict to a range; Specific Level restricts to an exact level (mutually exclusive with Min/Max). Leave all three blank for no restriction.

If no match at Specific Level (only enabled once Specific Level has a value) controls what Find Optimal Build does when nothing matches a wanted spell at exactly that level/tier:

- **Go down a tier** - keep the exact level, accept a lower tier if needed
- **Go down in level** - keep the exact tier, search downward for the closest available level
- **Both** - relax level and tier together, preferring the closest combination
- **Don't populate slot** (default) - no fallback; the slot is left empty

Only Found In (realm filter) - sits next to the Priority box, inside the "Basic" mini-tab. A small two-tab area of its own. The "Only Found In" tab holds Evil, Chaos, Good, Glory Bea, Crafted, Event, or Kaid checkboxes to restrict results to those realms. Leave everything unchecked (or check **All**) to search every realm except Crafted. Kaid has its own column: **Kaid All** matches any Kaid sub-realm, or check specific colors (White/Green/Red/Purple) instead - mutually exclusive with Kaid All. **Crafted items never appear unless the Crafted checkbox is checked**, and a build can include at most one Crafted-realm item even then. Below Crafted are two independent hard exclusions that apply no matter what else is selected above: **Non-Kaid** (excludes every Kaid realm) and **Non-Event** (excludes both Event and Glory Bea items). The second tab, **Events**, lists one checkbox per specific event found in the loaded database (e.g. "Halloween 2020", "Christmas 2023") - checking one lets that event's items through even when the broad Event checkbox above is left unchecked; both work together additively.

Save Constraints (to the right of Only Found In, above Required Items) - names and saves a full snapshot of every current Basic, Armor, Weapon, and Edibles Constraints selection at once, including which specific Events checkboxes are checked. Saved sets are listed below the button with **Load**, **Rename**, and **Delete** buttons - selecting an entry only highlights it, so an explicit Load click is needed to actually apply it (protecting against accidentally overwriting constraints you're still setting up). Saving under a name that already exists asks to confirm before overwriting it. Persists across closing and reopening the program.

3.3 Edibles

This is one of Build Constraints' own mini-tabs (see 3.2) - not a separate top-level sub-tab. Covers consumable items - anything with **Type = edible** in the master database - which a build can use any number of at once, unlike every other slot here which holds exactly one item.

- **Spell**/**tier** dropdowns and an **Add to List** button build an **Available Spells** whitelist, shown as removable chips - the spell dropdown's options are generated directly from whatever Type=edible items are actually in the loaded database (not a fixed list), and the tier dropdown only offers tiers that spell actually has among edible items (no "(any)" option - an entry always targets one specific tier). This list is independent

of the real Wanted Spells - an empty list means every edible is eligible; a non-empty one restricts to just edibles matching a listed entry exactly (base and tier both, no falling back to a different tier).

- **Add edibles to the build** checkbox - unchecked by default. Nothing here has any effect on a real search until this is checked.
- **Max Edibles Used** slider (1-13, snaps to whole numbers) - caps how many edibles a search will actually use at once.
- When enabled, Find Optimal Build picks up to that many edibles - the highest-level match per distinct spell among the eligible pool - and adds them to the build as extra **Stomach** rows in the Results tab. Show All Matches instead lists every eligible edible with no cap, matching that view's "show everything" philosophy.
- An edible's spell counts toward covering a real Wanted Spell exactly like an armor or weapon item would - if an edible already covers something, no other slot will redundantly carry the same spell at a different tier just because it happened to also be a good pick for its own slot.

3.4 Armor Constraints

This is one of Build Constraints' own mini-tabs (see 3.2) - not a separate top-level sub-tab.

- The **All:** row sets an armor type for every slot at once.
- Override individual slots (Head, Cloak, Body, Hands, Legs, Feet) with Cloth / Leather / Studded / Plate.
- Per-slot **Defense** (range) and **Sigil** dropdowns - soft preferences: a matching item is favored, but a slot is never left empty just because nothing matches.
- Up to 3 slots can be marked **Max Lvl** priority - greedily locks that slot to the highest-level item that still carries a wanted/priority spell, before the normal search runs for everything else.
- **Set as Default** / **Clear Default** / **Clear All** manage your saved preferences, persisted between sessions.

3.5 Weapon Constraints

Also one of Build Constraints' own mini-tabs (see 3.2) - not a separate top-level sub-tab.

- **Weapon Types/Combo's** - check whichever your build uses: Dual-Wield 1h, 1h/Shield, 2h/Shield, Two-Handed, 1 Claw, 2 Claw, Fired 1h/Shield. Most have their own Style (Melee/Direct/Parry Staff/Fired) and Damage Type (Slashing/Thrusting/Crushing) dropdowns. Each row sits in its own thin bordered box, alternating between two shades of grey row by row.
- Leave everything unchecked and Weapon/Shield aren't populated at all. 1 Claw/2 Claw replace Weapon and Shield entirely rather than filling alongside them.
- **Melee Weapon Constraints** - soft-preference Damage/Timer/Fumble/Accuracy/Sigil dropdowns (each with an optional Priority checkbox, capped at 3), plus a **Weight** range - soft preference by default, or check **Hard Filter** to exclude out-of-range weapons outright. Applies to every weapon style and to Claw slots too. 1 Claw/2 Claw each get their own Sigil dropdown (first for claw 1, second for claw 2).
- **Shield Constraints** - Defense and Sigil dropdowns working like Armor Constraints' per-slot versions, plus Leather/Studded/Plate checkboxes (a hard filter, same as Armor Constraints' own - Cloth isn't offered here since shields are never made of cloth in-game).

3.6 Bank Build

Paste a bank/inventory listing from the game and use it as the basis for a search, instead of (or alongside) the full database. Three inner tabs share the idea:

- **Import** - paste a listing, type a Character Name, then click Save to Saved Items. The first time a given name is used, a new tab for that character is created automatically under Character Items (or Lockers, if the Locker checkbox below is checked); using the same name again just updates that character's existing list. Matching is by spelling only, regardless of capitalization - typing "aria" or "ARIA" when "Aria" already exists updates that same character rather than creating a case-sensitive duplicate. This is the only way items get into a character's list now - Import doesn't run a search itself. A **Locker** checkbox is here too - a Locker isn't a character you play, it's extra bank space (a spare character made just to hold overflow gear). A Locker's non-Kaid gear is automatically folded into every other character's Bank Build search, regardless of that character's own Search all characters setting - unless its own tab has **Exclude this Locker from other characters' Bank Build searches** checked (unchecked by default), which opts just that one Locker out of everyone else's searches while still showing up normally on its own tab. Toggling this checkbox on a later Import for an existing character moves its tab between Character Items and Lockers automatically.
- **Character Items** - a notebook of its own. **Main** is a read-only combined view of every character's items (Lockers included), plus anything saved before character tracking existed - just the list, and a Clear Saved List button scoped to that older, character-less leftover data (it never touches any character's own items). **Each non-Locker character gets its own tab** with its own Only Found In checkboxes, Prioritize Saved Items/Hard Search checkboxes, a **Search all characters** checkbox, **Good Gear only**/**Invasion Gear only** checkboxes, a Find Best Bank Build button, a Clear Saved List button, and a **Delete** button (bottom-right) that permanently removes that character's entire tab and its saved items - unlike Clear Saved List, which only empties the list but keeps the tab around.
- **Lockers** - always the rightmost of the three inner tabs. Itself a small notebook holding one sub-tab per Locker character, each with the exact same controls as a regular character tab plus the Exclude-from-others checkbox and a note reminding you it's overflow storage, not a played character - keeps pure storage characters visually separate from ones you actually play. A leftmost **+** tab has no page of its own - clicking it prompts for a name and creates a new **Locker Group** tab to organize Locker tabs into. Drag a Locker tab onto a group tab to move it in, or drag it back out onto the main tab strip to ungroup it - group membership persists across restarts.

Hard Search holds each wanted spell's exact tier and restricts candidates to that character's items only - a slot with nothing at the exact tier reads "No available item". Since Hard Search only ever considers items you already have, it also ignores the Only Found In realm filter (Basic Constraints) - an owned item counts regardless of its original drop realm; every other search mode still respects Only Found In as usual. **Search all characters**, when checked, pools every character's non-Kaid items together for the search - but Kaid items still only come from that one character's own tab, since Kaid gear doesn't drop when you die and represents what that specific character is actually carrying, not the whole roster's stash. **Good Gear only**/**Invasion Gear only** restrict Find Best Bank Build to just items carrying that Gear Tag (see below) - both can be checked together (either tag qualifies), and neither checked runs unrestricted, as normal.

Every Character Items table (Main, every character, and every Locker) can be sorted by clicking any column header - click again to reverse. Level sorts numerically; everything else sorts as text.

Each row also has a **Gear Tag** column (the rightmost one) - click a cell for a dropdown of "Good Gear" / "Invasion Gear" / "Both" / "Blank". An untagged item defaults to **Good Gear** if it's an Event-realm item, or **Blank** otherwise - an explicit choice (including explicitly picking Blank) always overrides that default. One tag per item name, shared everywhere that item appears (Main and every character that has it), and persists across closing and reopening the program. For the Good Gear only/Invasion Gear only search checkboxes above, "Both" always qualifies under either one (it counts as both categories at once), while "Blank" qualifies under neither - so until you actually start tagging non-Event items, checking either box just filters the search down to whatever Event gear you already have.

Four paste formats are recognized and can even be mixed in one paste: a numbered Strongbox listing (*12.) item name [Level|Slot|...tags]*), the same listing without numbering, an Inventory listing ("(w)"/"(h)" marked lines always count; an unmarked or stack-count line also counts if it matches a real item in the loaded database), and an Items in use listing (*On Head: item name*, *Held Left: nothing*).

Each character's list reconciles by **content type** - bank listing vs. inventory/equipped listing - rather than which content was pasted most recently. Updating just a bank paste never wipes out that character's inventory-sourced items, and vice versa; a paste containing both kinds updates both. A second (or further) copy of the same item is listed at the bottom of a list, prefixed "::extra::". Everything persists across closing and reopening the program.

The Bank icon appears in the leftmost column of every search's results (Find Optimal Build, Show All Matches, everywhere) whenever an item matches something saved anywhere - any character, or the older Main-only leftover data. If the item instead came from a Locker, that column shows a bold "L" and the Results tab's Locker column (see 4) shows which one.

3.7 Manual

A standalone spell/tier database lookup, completely separate from the actual build search - nothing picked here ever feeds into Find Optimal Build/Show All Matches, and nothing from Basic/Armor/Weapon Constraints feeds into it either.

- Wanted Spell chips here are entirely optional - with none added, every item matching the other active constraints below shows up already; picking a spell from a category dropdown, an **exact** tier (no "any tier or higher" fallback - this is a precise lookup, not a build search), and clicking **Add to List** just narrows that down further to items whose Spell exactly matches. Add more than one spell/tier to see the combined (union) results.
- **Armor** / **Weapons** / **Jewel** checkboxes restrict the list to just the checked slot categories (checking none or all three means no restriction). Checking **Weapons** also adds Weight/Fumble/Damage/Timer/Accuracy columns to the results table. Actually using anything on the Armor Type mini-tab (a material, a per-slot "only this piece" checkbox, a per-slot Defense range, or a per-slot Sigil) counts as checking **Armor** too, even if you never checked it directly - otherwise those controls would only narrow which armor items show without ever actually excluding weapons/jewels/shields from the list.
- A **Level** range (min/max, blank side unbounded) - its own independent filter, separate from Basic Constraints' Level fields.
- **Only Found In** and **Armor Type**/**Weapon**/**Melee/Shield**/**Jewel** mini-notebooks sit beside the spell dropdowns - each is an independent copy of the same-named controls elsewhere (Only Found In's

Realms+Events, Armor Constraints' per-slot Defense/Sigil plus a simplified Cloth/Leather/Studded/Plate checklist, Weapon Constraints' Weapon Types/Combo's, Melee Weapon/Shield Constraints, and a Sigil dropdown for jewels). None of it is shared with the real tabs - it only ever affects Manual's own results.

- Armor Type's per-slot rows (Head/Cloak/Body/Hands/Legs/Feet) each have their own checkbox before the label too - checking one or more restricts the list to just those specific pieces, on top of whatever the broader Armor/Weapons/Jewel checkboxes above already allow.
- Right-click any row in the results table for **Add to Results Tab** - it adds into whichever Results view is currently showing (Best Per Slot, All Matches, or Manual's own flat list). For Best Per Slot/All Matches, an empty slot of the matching type is filled in place first if one's available; only once every empty slot of that type is already taken does the addition group as an extra row alongside it (so more than one item can occupy the same slot, e.g. 3 different body pieces - this is a curated addition, not a computed build). Manually-added gear survives Remove/Rebuild/Search Missing Slots elsewhere in the build, rather than only lasting until the next one of those recomputes the list. Right-click a manually-added row in the Results tab itself for **Remove**. Adding an item never switches away from the Manual tab.
- Click any column header in the results table to sort by it; click again to reverse.
- The results table leads with the same **Bank** (■) and **Locker** icon columns as the real Results tab - a plain ■ marks anything you've already saved anywhere, or a bold "L" plus the first 4 letters of the Locker's name for a Locker-sourced item. Purely informational here, since Manual has no build/scoring concept of its own to prioritize owned items within.

3.8 Search Controls

Min/Max/Specific Level, the "If no match at Specific Level" fallback options, and the three buttons below sit in the Wanted Spells column, right below **Clear All**. Since they're part of Basic Constraints' own layout, they're only visible while Basic Constraints is the selected sub-tab - not on Bank Build (which has its own per-character Find Best Bank Build button instead) or Manual (which has its own separate controls entirely - see 3.7).

- ■ **Find Optimal Build** - exact search across every slot at once for the combination covering as many wanted spells as possible under your constraints.
- ■ **Show All Matches** - lists every item matching your filters, without narrowing to one per slot.
- ■ **Generate multiple build options** - also generates up to 10 alternate full builds by swapping in equally-good tied items slot by slot.

4. Results Tab

- **Display: Best Per Slot / All Matches** toggles between the two search modes above.
- **Build Variant** dropdown - active when multiple build options were generated.
- Columns: Bank icon, **Locker**, Slot, Item, Type, Spell, Sigil, Level, Mob, Area, and **Alt Options** (other items that tied for that slot). The Locker column shows the first 4 letters of a Locker character's name whenever a row's item came from one - the Bank column shows a bold "L" instead of the usual ■ for those rows. Edibles show as **Stomach** in the Slot column.

- Every slot the search actually tried to fill shows a row, even if it stays empty ("No suitable item found") - this no longer depends on whether some other slot happened to leave a wanted spell uncovered.
- A row added via the Manual tab (see 3.7) shows "Manual Input" in Alt Options - right-click it for a simple **Remove**, distinct from the options below (which resolve back to a real build slot).
- **Right-click a filled item** to remove it from the build - the slot stays visible showing "No suitable item found" rather than disappearing, and it stays excluded from every future search until a genuinely new search runs. Two buttons appear once something's been removed:
- **Rebuild (Full Database)** - recomputes the whole build, re-searching everywhere, excluding what's been removed.
- **Rebuild (Saved Items First)** - builds as much of the whole set as possible from Saved Items (Lockers' non-Kaid gear included, their Kaid items excluded), then automatically searches the full database for whatever it can't cover - a complete gear set every time. Useful for PvP: remove a good item you don't want to risk losing on death, rebuild, and get a full set that reuses your bank wherever possible and clearly shows (via the missing Bank icon) exactly what you'd still need to go acquire for the rest.
- **Right-click an empty slot** ("No suitable item found", etc.) offers three options: **Search Full Database for This Slot** (finds a single best-fit item for just that one slot, honoring every active constraint and correctly avoiding a wanted spell some other slot already covers, without touching or recomputing anything else in the build); **Rebuild (Saved Items First)** (the same full-build rebuild described above); and **Rebuild (Full Database, Prefer Owned)** - a single unrestricted search that still favors items you already own, generating a stack of build options in the Build Variant dropdown ("0 new items", "1 new item", "2 new items", etc.) - the fewest new items needed to reach each successively better result, so you can weigh a smaller shopping list against a stronger build. When a looser cap has spare budget left over that coverage alone doesn't need, it also adds "N new items (alt)" options - equally-good unowned alternatives swapped into an owned slot one at a time, just for extra choices to compare.
- **Search Missing Slots (Full Database)** button appears whenever a Saved Items Hard Search leaves gaps - fills them in place from the full database, asking about a lower tier if the exact one isn't found anywhere.
- **Remove Area** - strip all items from a given area out of the current results.
- **Export Results** / **Save Build** - save the current table to Excel, HTML, image, or text, or pin it as a permanent panel under Saved Builds.

5. Saved Builds Tab

- Click **Save Build** from the Results tab to add the current results as a new sub-tab here - a Notebook, not a stacked scrollable list, so having many saves just adds more tabs rather than a taller page.
- Each sub-tab has its own renamable name (also shown right on the tab label itself), a **Load** button (copies that build's rows into the Results tab and jumps there), an **Export As...** (Excel/HTML/Image/Text), and a **Remove** button.
- Saved Builds persist across closing and reopening the program.

6. Area Items Tab

- Pick an Area and browse every item droppable there, straight from the loaded master database.
- The Area field is typeable - type a few letters to narrow the list, or click/tab into it to see the full alphabetical list right away.
- Results are sorted by slot: armor first (Plate, Studded, Leather, Cloth), then Jewel, Shield, Weapon. Click any column header to sort by that column instead; click again to reverse.

7. Tips and Example Builds

Quick start: Use Community List, add the spells you want, set level/armor/weapon/realm constraints as needed, click Find Optimal Build.

- **Tank:** Melee + Plate armor + Weapon + Shield + Slashing + Min Level 60
- **Two-Handed Warrior:** Melee + Plate armor + Two-Handed + Crushing + Level 50-70
- **Dual-Wield Rogue:** Melee + Leather armor + Weapon + Dual-Wield + Thrusting
- **Dual-Claw Fighter:** Melee + 2 Claw + Slashing
- **Caster with Staff:** Direct (Caster) + Cloth armor + Two-Handed + Crushing

8. Troubleshooting

- **"Windows protected your PC" / SmartScreen warning** - expected, the exe isn't code-signed yet. Click "More info" then "Run anyway".
- **The exe takes a few seconds to open every time** - expected for the single-file download; it's a PyInstaller onefile build that unpacks itself to a temp folder on every launch. Use the Folder download instead (see 1. Getting Started) for instant startup with no unpacking step.
- **Files not loading** - check the file extension (.log, .txt, .xlsx), make sure it isn't open in another program, and try copying it somewhere else if the path has unusual characters.
- **Parse results empty** - verify the log actually contains delve output ("You examine X closely") and that CHAT vs ACTION auto-detection picked the right type.
- **Community List not found** - if running from source rather than the .exe, make sure Olmran_Community_Eq_and_Stats_List.xlsx is in the same folder, or use Browse... to point at it manually.
- **Can't see everything if the window is resized smaller** - the whole window scrolls (mouse wheel, or the scrollbar on the right edge) if it's ever shrunk below what its content needs.

Full version history and the latest release are on GitHub: github.com/Gnawbie/Olmran-parser-ItemBuilder